Step 3 — A Retriever Agent That Judges Its Own Search Results — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: the first searching agent. The Retriever searches your documents with Doc08's retriever — and then **judges** what it found against a written-down relevance bar, instead of passing along "the closest thing" no matter how far off it is. A question with no real match goes to an honest "not found" path; questions that aren't about the documents skip the search. The Reasoner's file and node also appear here, starting simple: it quotes the best excerpt. **If not:** every question would get an answer built on whatever the search happened to return.

After this step document questions are answered from your documents (still approved by you). `answer_node`, `approval_node` and `main.py` don't change. Files not shown here don't change.

The RAG files

Story — `rag/chunking.py`, `rag/ingest.py`, `rag/retriever.py`: Doc08's Build Task (Intermediate, Approach 2), with two small changes: the OpenAI client is created *inside* each function (so the key from `.env` is loaded first, like the logger), and `build_vector_store()` logs what it ingested. **If not:** you'd rewrite chunking, embedding and search from scratch — and lose Doc08's safe delete-then-rebuild.

```
# src/documind/rag/chunking.py
# from Doc08 — two named, swappable ways to split a document

def chunk_by_chars(text: str, chunk_size: int = 300) -> list[str]:
    chunks: list[str] = []
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i += chunk_size
    return chunks

def chunk_by_paragraph(text: str) -> list[str]:
    raw_pieces = text.split("\n\n")
    chunks = []
    for piece in raw_pieces:
        stripped = piece.strip()
```

```
        if stripped != "":
            chunks.append(stripped)
    return chunks
```

```
# src/documind/rag/ingest.py
from pathlib import Path
from typing import Callable

import chromadb
from openai import OpenAI

from documind.rag.chunking import chunk_by_paragraph
from documind.utils.logger import get_logger

EMBEDDING_MODEL = "text-embedding-3-small"

chroma_client = chromadb.Client()

def build_vector_store(
    doc_folder: str,
    chunk_fn: Callable[[str], list[str]] = chunk_by_paragraph,
    collection_name: str = "docs",
):
    logger = get_logger(__name__)
    # when: the OpenAI client is made here, not at import time, so the
    # key from .env is already loaded by load_config()
    client = OpenAI()
    # why: delete-then-recreate starts every run from a clean, known state
    # (Doc08) — a re-run never duplicates chunks
    try:
        chroma_client.delete_collection(name=collection_name)
    except Exception:
        pass  # how: didn't exist yet — fine, this is the first run
    collection = chroma_client.create_collection(name=collection_name)

    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    all_ids: list[str] = []

    for path in Path(doc_folder).glob("*"):
        if path.suffix not in (".txt", ".md"):
            continue
        text = path.read_text()
        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
```

```
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
            })
            # why: deterministic, not a random UUID
            all_ids.append(f"{path.name}-{chunk_index}")

    if not all_texts:
        logger.warning("no documents found in %s", doc_folder)
        return collection

    # why: one batched call for every chunk across every file
    response = client.embeddings.create(model=EMBEDDING_MODEL, input=all_texts)
    embeddings = []
    for item in response.data:
        embeddings.append(item.embedding)

    collection.add(
        ids=all_ids, embeddings=embeddings,
        documents=all_texts, metadatas=all_metadatas,
    )
    logger.info("ingested %s chunks from %s", len(all_texts), doc_folder)
    return collection
```

```
# src/documind/rag/retriever.py
from openai import OpenAI

from documind.rag.ingest import EMBEDDING_MODEL

def retrieve(collection, query: str, k: int = 3) -> list[dict]:
    if collection.count() == 0:
        return []  # when: an empty store must not crash (Doc08)

    # when: made here, after load_config() has loaded the key
    client = OpenAI()
    response = client.embeddings.create(model=EMBEDDING_MODEL, input=[query])
    query_embedding = response.data[0].embedding

    results = collection.query(query_embeddings=[query_embedding], n_results=k)

    texts = results["documents"][0]
    metadatas = results["metadatas"][0]
    distances = results["distances"][0]

    documents: list[dict] = []
    for text, metadata, distance in zip(texts, metadatas, distances):
        documents.append({
```

```
            "text": text,
            "source_file": metadata["source_file"],
            "chunk_index": metadata["chunk_index"],
            # how: a distance — LOWER means a closer match
            "score": distance,
        })

    return documents
```

Story — `rag/store.py`: builds the vector store **once**, the first time a node needs it, and keeps it — Doc07's small "state holder" class pattern. **If not:** building at import time would run before the key is loaded, and building in the node would re-embed every document for every question.

```
# src/documind/rag/store.py
from documind.rag.ingest import build_vector_store

DOCS_FOLDER = "data/docs"

class StoreHolder:
    def __init__(self) -> None:
        # why: the vector store is built once, the first time a node needs
        # it — not at import time, and not again for every question
        self.collection = None

store = StoreHolder()

def get_collection():
    if store.collection is None:
        store.collection = build_vector_store(DOCS_FOLDER)
    return store.collection
```

Also create an empty `rag/__init__.py`, and write your documents into `data/docs/` — 5 short `.md` files, each a few paragraphs, **no heading lines** (a heading alone becomes its own useless chunk). For example:

`data/docs/refund_policy.md`

```
Customers can return any product within 30 days of purchase for a full
refund. The original receipt or order number is required.

Refunds are paid back to the original payment method within 5 working
days after the returned product arrives at our warehouse.
```

The other four documents in the example set:

`data/docs/shipping.md`

```
Orders are shipped within 2 working days. Standard delivery takes
3 to 5 working days, and express delivery takes 1 to 2 working days.

Shipping is free on orders over 50 dollars. Every order gets a
tracking number by email as soon as it leaves our warehouse.
```

`data/docs/warranty.md`

```
Every product has a 1-year warranty against manufacturing faults.
The warranty starts on the day the product is delivered.

The warranty does not cover damage from drops, water or misuse. To
make a claim, send the order number and a photo of the fault.
```

`data/docs/account_security.md`

```
Passwords must be at least 12 characters long. Support staff will
never ask for your password, by email, chat or phone.

After 5 failed login attempts, the account is locked for 15 minutes.
Two-factor authentication can be turned on in the account settings.
```

`data/docs/pricing.md`

```
The Basic plan costs 9 dollars a month and the Pro plan costs 29
dollars a month. Both plans can be cancelled at any time.

Paying yearly gives two months free. Prices include tax for customers
in the European Union.
```

Story — `agents/reasoner_agent.py`: the Reasoner's file and node, starting as simply as possible: `reason_node` answers with the best excerpt, word for word, plus its file name — no model yet. Step 4 keeps this node and upgrades what's inside it. **If not:** a model-written answer would arrive at the same time as the search, and a bad answer couldn't be pinned on either one.

```
# src/documind/agents/reasoner_agent.py
from documind.state import DocState

def reason_node(state: DocState) -> dict:
    # why: a simple, honest answer for now — the best matching excerpt,
    # word for word, with its file name. Step 4 keeps this node and
```

```
    # upgrades it to write a real answer from ALL the excerpts.
    best = state["sources"][0]
    answer = best["text"] + " [" + best["source_file"] + "]"
    return {"answer": answer, "path": ["reason_node"]}
```

Back to this step · Hint 1 · Hint 2 · Solution

Solution — a written relevance bar, and an honest "not found" path

Changed from Step 2: `state.py` adds `sources` and `grounded`.

Story — `state.py`: the found chunks live **in the state**. A run paused for approval keeps its evidence, and resuming never searches again. **If not:** a resumed run would have its answer but no record of where it came from.

```
# src/documind/state.py
import operator
from typing import Annotated, TypedDict

class DocState(TypedDict):
    # why: one typed shape listing everything any node can read or write
    task: str
    # how: operator.add appends each node's entries instead of replacing
    # the list — so path is a running log of the nodes that ran (Doc09)
    path: Annotated[list[str], operator.add]
    sources: list[dict]
    grounded: bool
    approved: bool
    answer: str

def start_state(task: str) -> DocState:
    # why: every field starts with a known value, so no node ever has to
    # guess whether a key exists yet
    return {
        "task": task,
        "path": [],
        "sources": [],
        "grounded": False,
        "approved": False,
        "answer": "",
    }
```

Story — `agents/retriever_agent.py`: four small functions. `route_search()` decides search-or-skip by keyword (Doc10). `is_relevant()` is the relevance bar: the **best** distance must be at most `MAX_DISTANCE` (lower = closer, Doc08). `retrieve_node()` searches and records the verdict. `no_grounding_node()` gives the honest answer. **If not:** "grounded" would mean "the search returned something" — which is always true.

```
# src/documind/agents/retriever_agent.py
from documind.rag.retriever import retrieve
from documind.rag.store import get_collection
from documind.state import DocState
from documind.utils.logger import get_logger

# why: questions about these topics are searched in the documents;
# anything else is answered from general knowledge (Doc10's routing)
SEARCH_KEYWORDS = [
    "document", "policy", "refund", "return", "shipping", "delivery",
    "warranty", "password", "account", "price", "plan",
]

# why: the relevance bar, written down BEFORE testing — a chunk counts as
# a real match only if its distance is at most this. Lower = closer.
# how: print a few scores for your own documents to tune it (Doc08)
MAX_DISTANCE = 1.2

def route_search(state: DocState) -> str:
    # how: a conditional edge — returns the name of the next branch
    task = state["task"].lower()
    for keyword in SEARCH_KEYWORDS:
        if keyword in task:
            return "search"
    return "skip"

def is_relevant(sources: list[dict]) -> bool:
    # why: "the search returned something" is not the same as "the search
    # found the answer" — check the best score against the bar
    if not sources:
        return False
    best = sources[0]["score"]
    for source in sources:
        if source["score"] < best:
            best = source["score"]
    return best <= MAX_DISTANCE

def retrieve_node(state: DocState) -> dict:
    logger = get_logger(__name__)
    sources = retrieve(get_collection(), state["task"], k=3)
    grounded = is_relevant(sources)
    for source in sources:
        logger.info("found %s (score %.2f)",
                    source["source_file"], source["score"])
    # why: the found chunks go INTO the state, so a paused run keeps its
    # grounding and never has to search again after resuming
    return {"sources": sources, "grounded": grounded,
```

```
            "path": ["retrieve_node"]}

def has_grounding(state: DocState) -> str:
    if state["grounded"]:
        return "reason"
    return "no_grounding"

def no_grounding_node(state: DocState) -> dict:
    # why: an honest "not found" instead of an answer built on nothing
    answer = "I couldn't find this in your documents, so I won't guess."
    return {"answer": answer, "path": ["no_grounding_node"]}
```

Changed from Step 2: two conditional edges and three new nodes. General answers and "not found" end the run; only a document answer goes to `approval_node`. This is the graph's **final** shape — Steps 4 and 5 don't change this file.

Story — `graph.py`: `add_conditional_edges(node, function, map)` — the function's return value picks the next node (Doc09). **If not:** every path would look the same, and a "not found" answer would wait for approval for no reason.

```
# src/documind/graph.py
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph

from documind.agents.approval_agent import approval_node
from documind.agents.reasoner_agent import reason_node
from documind.agents.retriever_agent import (
    has_grounding,
    no_grounding_node,
    retrieve_node,
    route_search,
)
from documind.nodes import answer_node, clean_task
from documind.state import DocState

def build_graph(checkpointer=None):
    # why: the checkpointer is passed in (Doc09) — main.py uses a SQLite
    # one that survives a restart, the tests use a fresh MemorySaver
    if checkpointer is None:
        checkpointer = MemorySaver()

    builder = StateGraph(DocState)
    builder.add_node("clean_task", clean_task)
    builder.add_node("answer_node", answer_node)
    builder.add_node("retrieve_node", retrieve_node)
    builder.add_node("no_grounding_node", no_grounding_node)
```

```
    builder.add_node("reason_node", reason_node)
    builder.add_node("approval_node", approval_node)

    builder.add_edge(START, "clean_task")
    # how: a conditional edge — route_search's return value picks the
    # next node from this map
    builder.add_conditional_edges(
        "clean_task", route_search,
        {"search": "retrieve_node", "skip": "answer_node"},
    )
    builder.add_conditional_edges(
        "retrieve_node", has_grounding,
        {"reason": "reason_node", "no_grounding": "no_grounding_node"},
    )
    builder.add_edge("reason_node", "approval_node")
    builder.add_edge("approval_node", END)
    builder.add_edge("answer_node", END)
    builder.add_edge("no_grounding_node", END)
    return builder.compile(checkpointer=checkpointer)
```

Story — `tests/unit/test_chunking.py` + `tests/unit/test_routing.py`: check the chunkers and the routing/relevance rules with hand-made inputs — no API key. **If not:** a wrong `MAX_DISTANCE` comparison (`>` instead of `<=`) would only show up as strange answers.

```
# tests/unit/test_chunking.py
# Checks Doc08's two chunkers. No API key needed.
# Run from the repo root: python tests/unit/test_chunking.py
from documind.rag.chunking import chunk_by_chars, chunk_by_paragraph

def test_chunk_by_chars() -> None:
    chunks = chunk_by_chars("a" * 650, chunk_size=300)
    lengths = []
    for chunk in chunks:
        lengths.append(len(chunk))
    print(f"case 1 (650 chars, size 300) -> {lengths}")

def test_chunk_by_paragraph() -> None:
    text = "First paragraph.\n\n\n\nSecond paragraph.\n\n   \n\n"
    print(f"case 2 (paragraphs)          -> {chunk_by_paragraph(text)}")

if __name__ == "__main__":
    test_chunk_by_chars()
    test_chunk_by_paragraph()
```

```
# tests/unit/test_routing.py
# Checks the Retriever agent's routing and relevance rules with
# hand-made states. No API key needed.
# Run from the repo root: python tests/unit/test_routing.py
from documind.agents.retriever_agent import (
    has_grounding,
    is_relevant,
    route_search,
)

def test_route_search() -> None:
    doc_task = {"task": "What is the refund policy?"}
    general_task = {"task": "What is the capital of France?"}
    print(f"case 1 (refund question)   -> {route_search(doc_task)}")
    print(f"case 1 (general question)  -> {route_search(general_task)}")

def test_is_relevant() -> None:
    close = [{"score": 1.6}, {"score": 0.4}]
    far = [{"score": 1.7}, {"score": 1.9}]
    print(f"case 2 (best score 0.4)    -> {is_relevant(close)}")
    print(f"case 2 (best score 1.7)    -> {is_relevant(far)}")
    print(f"case 2 (nothing found)     -> {is_relevant([])}")

def test_has_grounding() -> None:
    print(f"case 3 (grounded=True)     -> {has_grounding({'grounded': True})}")
    print(f"case 3 (grounded=False)    -> "
          f"{has_grounding({'grounded': False})}")

if __name__ == "__main__":
    test_route_search()
    test_is_relevant()
    test_has_grounding()
```

Story — `tests/integration/test_live_graph.py`: the three paths against the real API: a document question (paused for approval, then approved), a question the documents don't cover, and a general question. **If not:** "the not-found path works" would be a hope.

```
# tests/integration/test_live_graph.py
# Runs the whole graph against the real OpenAI API.
# Run from the repo root: python tests/integration/test_live_graph.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from documind.config import load_config
from documind.graph import build_graph
```

```
from documind.state import start_state

def run(graph, thread_id: str, task: str) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    return graph.invoke(start_state(task), config)

def test_grounded_answer(graph) -> None:
    # case 1: a document question — searched, drafted, paused for review
    paused = run(graph, "t1", "What is the refund policy?")
    print(f"case 1 (paused?)      -> {'__interrupt__' in paused}")
    config = {"configurable": {"thread_id": "t1"}}
    result = graph.invoke(Command(resume="yes"), config)
    print(f"case 1 (grounded)     -> {result['path']}")
    print(f"                         {result['answer']}")

def test_not_in_documents(graph) -> None:
    # case 2: searched, but nothing relevant — must not make one up
    result = run(graph, "t2", "What is the policy on remote work?")
    print(f"case 2 (not found)    -> {result['path']}")

def test_general_question(graph) -> None:
    # case 3: not about the documents — no search at all
    result = run(graph, "t3", "What is the capital of France?")
    print(f"case 3 (general)      -> {result['path']}")

if __name__ == "__main__":
    load_config()
    # how: tests use a fresh MemorySaver — nothing is written to disk
    graph = build_graph(MemorySaver())
    test_grounded_answer(graph)
    test_not_in_documents(graph)
    test_general_question(graph)
```

Run everything:

```
python tests/unit/test_chunking.py
python tests/unit/test_routing.py
python tests/integration/test_live_graph.py
python -m documind.main
```

Expected integration output:

```
case 1 (paused?)      -> True
case 1 (grounded)     -> ['clean_task', 'retrieve_node', 'reason_node',
                          'human_approved']
                         Customers can return any product within 30 days
```

```
                          ... [refund_policy.md]
case 2 (not found)    -> ['clean_task', 'retrieve_node', 'no_grounding_node']
case 3 (general)      -> ['clean_task', 'answer_node']
```

(Long lines are wrapped here to fit the page.) To tune `MAX_DISTANCE` for your own documents, watch the `found ... (score ...)` log lines for a few good and bad questions, and set the bar between them.